

QuantumSemEval: Benchmarking LLMs’ Understanding of Quantum Programs via Semantic Equivalence Checking

Anonymous Authors¹

Abstract

Large language models (LLMs) have shown promise in reasoning about code, yet their ability to perform semantic reasoning over quantum programs remains largely unexplored. To address this gap, we introduce **QuantumSemEval**, a benchmark designed to evaluate LLMs’ capacity to determine semantic equivalence between pairs of quantum programs, comprising 260 labeled pairs (130 equivalent and 130 non-equivalent) across diverse algorithms. We evaluate thirteen LLMs spanning proprietary and open-weight architectures. While models reliably detect non-equivalence, identifying semantically equivalent circuits is substantially more challenging. Top performance on equivalent pairs reaches only 70.8%, with the majority of models performing below the 50% random baseline. These results highlight the current limitations of LLMs in semantic reasoning and provide a foundation for future work aimed at enhancing their understanding of quantum programs.

1. Introduction

Can large language models (LLMs) truly understand the behavior of a quantum program? The relevance of this question has grown as quantum computing moves beyond laboratory research into practical, real-world applications. Researchers are leveraging quantum computing for combinatorial optimization (Shikuri, 2025; Moncayo-Martínez & He, 2025; Calude et al., 2020), machine learning and feature selection (Mücke et al., 2023; Pathak et al., 2025; Samanta et al., 2025), scheduling and logistics (Zhou et al., 2025; Ciacco et al., 2026; Gebler et al., 2024), cybersecurity (Marini & Arpe, 2025), and simulating molecular systems in chemistry (Negre et al., 2022).

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

As quantum computing applications continue to expand, there is an increasing demand for tools that can facilitate the development and verification of quantum programs. Large language models (LLMs) have emerged as a promising candidate to assist in this process. However, before deploying LLMs for such tasks, it is critical to assess whether they genuinely comprehend the semantics of quantum programs, rather than merely reproducing patterns encountered during their training.

This challenge is non-trivial because quantum programs operate very differently from classical programs. In a classical program, instructions execute one after another in a predictable, deterministic manner. By contrast, quantum programs run on quantum computers that leverage principles of quantum mechanics, and their outputs are inherently probabilistic. Even small modifications to a quantum program may either leave its behavior unchanged or produce substantially different results. The difficulty is further compounded by noise and imperfections in current quantum hardware, which is still in the initial stage known as the noisy intermediate-scale quantum (NISQ) era (Preskill, 2018). As a result, repeated executions of the same quantum program may yield different outcomes making it difficult to tell whether differences in the outputs of quantum programs are due to actual changes in their behavior or just noise from the hardware, making it a challenging task even for experts.

Most existing benchmarks for quantum programming focus on whether individual programs run correctly and produce expected outputs. They do not evaluate whether two programs, which may appear different, actually compute the same function. Classical equivalence benchmarks perform this type of comparison, but they do not account for the unique features of quantum programs. Consequently, there is currently no systematic method to assess how well LLMs reason about the meaning of quantum programs, despite their increasing use in the field.

To address this gap, we introduce **QuantumSemEval**, a benchmark designed to evaluate LLMs’ ability to determine when two quantum programs are semantically equivalent.

Our contributions are as follows:

- We introduce **QuantumSemEval**, a benchmark specifically designed to evaluate large language models (LLMs) on the semantic equivalence of quantum programs which comprises 260 circuit pairs (130 equivalent and 130 non-equivalent) covering a diverse set of quantum algorithms, transformations, and ZX-calculus optimizations.
- We evaluate thirteen LLMs spanning proprietary and open-weight architectures, rigorously comparing their reasoning performance on both equivalent and non-equivalent circuits. The results reveal a pronounced asymmetry between tasks: confirming semantic equivalence is substantially more challenging than detecting non-equivalence. The top performers on equivalent pairs are o4-mini (70.8%) and Grok 4.3 (70.0%), while the top performers on non-equivalent pairs are GPT-OSS 120B (100%) and Gemini 2.5 Pro (99.2%), clearly highlighting that semantic equivalence detection is the more difficult task than non-equivalence detection for current LLMs.

2. Related Work

Equivalence checking has its roots in classical program analysis. Datasets like EQBENCH (Badihi et al., 2021), SeqCoBench (Maveli et al., 2025), and more recently EquiBench (Wei et al., 2025) evaluate whether two classical programs produce the same outputs across all inputs. These benchmarks mainly cover optimization, refactoring, code transpilation, and testing, often using small datasets or relying on statement-level syntactic changes such as variable renaming. While EQBENCH and SeqCoBench provide labeled program pairs for equivalence checking, they were not specifically designed to assess large language models. In contrast, EquiBench explicitly benchmarks LLMs’ capability to reason about program semantics, providing a large-scale and systematic evaluation of model performance on equivalence tasks.

Extending beyond classical programs, recent research has begun to systematically evaluate the ability of large language models (LLMs) to generate correct and executable quantum programs. One of the earliest dedicated benchmarks for quantum code generation is Qiskit HumanEval (Vishwakarma et al., 2024), which introduces a hand-curated dataset of over 100 quantum computing tasks written using the Qiskit SDK. Each task includes a canonical solution and functional test cases, and the benchmark measures LLM performance on producing executable quantum code.

Following this, QuanBench (Guo et al., 2025) was proposed as a broader quantum code generation benchmark with 44 tasks covering quantum algorithms, state preparation, gate decomposition, and quantum machine learning. Tasks are evaluated by functional correctness (Pass@K) as well as

semantic equivalence metrics such as process fidelity, highlighting the challenges LLMs face in generating semantically valid quantum code.

More recently, QuanBench Plus (Slim et al., 2026) extends this evaluation to a unified multi-framework setting by aligning tasks across three major quantum computing frameworks: Qiskit, PennyLane, and Cirq. QuanBench+ assesses models with executable functional tests and reports Pass@1 and Pass@5 scores across frameworks, including feedback-based repair strategies to improve model outputs.

Another emerging benchmark, QHackBench (Basit et al., 2025), evaluates LLMs on quantum code generation using real quantum programming challenges derived from PennyLane hackathon data. QHackBench assesses syntactic validity, execution success, and functional correctness under both standard prompting and retrieval-augmented generation pipelines.

Existing benchmarks primarily focus on generating syntactically correct quantum code or assessing execution success for isolated tasks (Vishwakarma et al., 2024; Guo et al., 2025; Slim et al., 2026; Basit et al., 2025). To the best of the authors’ knowledge, no benchmark systematically evaluates large language models on the **semantic equivalence of quantum programs**, leaving a critical gap: while classical equivalence datasets provide useful foundations and quantum code generation benchmarks measure correctness, the semantic equivalence of quantum programs remains unexplored.

3. Quantum Programs and Semantic Equivalence

3.1. Quantum Unit of Information: The Qubit

Classical computers process information using bits. A bit can have only one of two values at any time: 0 or 1. Quantum computers use *quantum bits*, or *qubits*, which are different because they can exist in a combination of 0 and 1 simultaneously. Here, 0 and 1 are represented as column vectors, and this behavior relies on the quantum mechanical principle of superposition. Using Dirac notation, the computational basis states are written as

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A general qubit state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where $\alpha, \beta \in \mathbb{C}$ are complex numbers called *amplitudes*, satisfying

$$|\alpha|^2 + |\beta|^2 = 1.$$

When the qubit is measured, it produces a classical result: 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$ (Nielsen & Chuang, 2010; Wong, 2022).

3.2. Quantum Information Processing

Quantum information is stored in qubits, which can exist in linear combinations of $|0\rangle$ and $|1\rangle$ states. These qubits are modified using operators represented by *unitary matrices* U . Applying these matrices changes the direction of the column vectors $|0\rangle$ and $|1\rangle$, effectively modifying the information stored in the qubit.

These matrices are called *quantum gates*, and they satisfy the unitary property:

$$U^\dagger U = I,$$

where $U^\dagger$ is the conjugate transpose of U and I is the identity matrix. This property ensures that the total probability of all outcomes remains 1 and that the transformation is reversible.

Formally, applying a quantum gate U to a qubit in state $|\psi\rangle$ produces a new state:

$$|\psi'\rangle = U|\psi\rangle.$$

Some common examples of quantum gates include:

1. Pauli-X gate (quantum analogue of NOT):

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 * 1 + 1 * 0 \\ 1 * 1 + 0 * 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle$$

2. Hadamard gate (creates superposition):

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

3.3. Quantum Program Semantics and Equivalence

For this paper, we view a quantum program as an object with mathematical semantics. In the simplest case, when there is no intermediate measurement or noise, a quantum circuit denotes a unitary matrix. If a circuit C contains gates $U_1, U_2, \dots, U_T$, then its overall unitary semantics is

$$U_C = U_T U_{T-1} \cdots U_1.$$

The order is important because matrix multiplication is generally not commutative.

Two measurement-free quantum circuits C_1 and C_2 are equivalent if they apply the same transformation to every possible input state:

$$U_{C_1}|\psi\rangle = U_{C_2}|\psi\rangle \quad \text{for all valid quantum states } |\psi\rangle.$$

Because a global phase cannot be observed physically, we may also treat two circuits as equivalent when

$$U_{C_1} = e^{i\theta} U_{C_2}$$

for some real number θ (Peham et al., 2022b).

When measurement is included, equivalence can be defined in terms of output probabilities. Two quantum programs P_1 and P_2 are semantically equivalent if for every input x and every possible measured output y ,

$$\Pr[P_1(x) = y] = \Pr[P_2(x) = y].$$

This definition is close to the classical idea of program equivalence, but the output of a quantum program is usually a probability distribution rather than a single deterministic value.

4. Benchmark Construction

QuantumSemEval is designed to evaluate large language models (LLMs) on the **semantic equivalence of quantum programs**. The benchmark comprises pairs of quantum circuits represented using **Qiskit**, a Python framework for quantum computing (Javadi-Abhari et al., 2024), categorized as either semantically *equivalent* or *non-equivalent*, and encompassing a diverse set of quantum algorithms and transformations. The dataset contains a total of 130 equivalent pairs and 130 non-equivalent pairs. Both types of pairs are generated and curated automatically using established frameworks and deterministic transformation functions. In the following subsections, we provide a detailed description of the dataset construction and methodology.

4.1. Equivalent Pairs

Equivalent circuit pairs are generated by applying **ZX-calculus optimizations** using the PyZX framework (Kissinger & Van De Wetering, 2019). Each pair consists of:

- **Original circuit:** A known quantum algorithm, sourced from canonical solution circuits in Qiskit HumanEval (Vishwakarma et al., 2024), the QunBench benchmark (Guo et al., 2025), publicly available GitHub implementations of textbook algorithms, and randomly generated circuits produced via Qiskit. Figure 1 shows the distribution of the types of source circuits used in the benchmark.

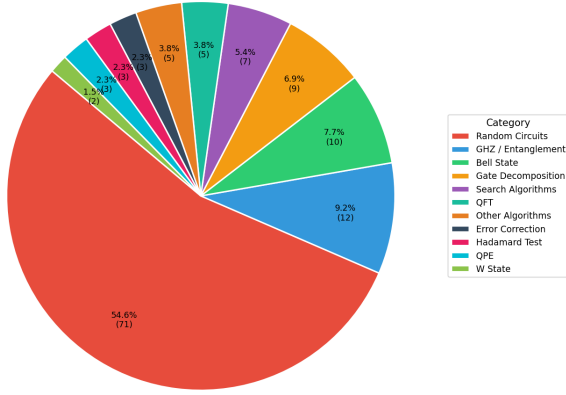


Figure 1. Type distribution of the source circuits in the benchmark. The dataset comprises standard textbook algorithms (e.g., Bernstein-Vazirani, Grover, GHZ) (Nielsen & Chuang, 2010) and randomly generated circuits included for stress testing, as the textbook algorithms may have been seen by LLMs during training.

Category	Original length	ZX length
Random Circuits	38.9	159.4
GHZ / Entanglement	5.2	9.5
Bell State	4.0	7.8
Gate Decomposition	4.9	26.8
Search Algorithms	11.3	17.0
QFT	8.0	17.4
Other Algorithms	3.2	16.0
Error Correction	7.3	11.3
Hadamard Test	7.0	31.0
QPE	13.0	23.3
W State	6.0	15.5

Table 1. Average lines of code per task category before and after ZX-calculus transformation.

- **Optimized circuit:** The original circuit is simplified using ZX-calculus (`full_reduce` in PyZX) and then converted back into an equivalent Qiskit representation. An illustrative example of ZX transformation is shown in Figure 2. The semantic equivalence of the pair is verified by computing the **statevector fidelity**, which measures the closeness of two quantum states and is defined as $F(\psi, \phi) = |\langle \psi | \phi \rangle|^2$ (Nielsen & Chuang, 2010), ensuring a fidelity of 1 for equivalent circuits. As shown in Table 1, the ZX-calculus transformation substantially expands code length across all categories, confirming that the two semantically equivalent circuit representations are syntactically dissimilar.

4.2. Non-Equivalent Pairs

Non-equivalent circuit pairs are generated by applying targeted **semantic-breaking mutations** to source circuits drawn from the same pool as the equivalent set. Each mutated circuit is verified to be non-equivalent by computing the statevector fidelity $F(\psi, \phi) = |\langle \psi | \phi \rangle|^2$ (Nielsen & Chuang, 2010) and confirming $F < 1$ for at least one computational basis input state. The mutations are designed to be *structurally subtle* and the mutated circuit retains nearly identical gate depth to the original. The non-equivalent dataset nearly mirrors the algorithmic distribution of the equivalent set, ensuring a consistent evaluation landscape across both conditions.

Seven mutation strategies are employed, as illustrated in Figure 3:

- **Reorder (31 pairs):** Gates that appear commutative are deliberately resequenced e.g., moving a gate before or after a non-commuting operation. Although the circuit looks syntactically similar, the altered execution order changes the resulting unitary.
- **Insertion (21 pairs):** One additional gate (e.g., Z , S) is inserted at a semantically critical position, increasing circuit depth marginally while breaking equivalence.
- **Single Deletion (11 pairs):** One gate is removed from the circuit.
- **Double Deletion (10 pairs):** Two gates are removed simultaneously, while still preserving the overall circuit topology.
- **Single Substitution (21 pairs):** One gate is replaced by a semantically distinct gate of the same arity (e.g., $cX \rightarrow cY$), leaving depth unchanged and making the mutation visually hard to detect.
- **Double Substitution (21 pairs):** Two gates are simultaneously replaced, compounding the single-substitution challenge while maintaining circuit depth.
- **CX Swap (15 pairs):** The control and target qubits of one or all CX gates are swapped. Since the CX gate is not symmetric, this fundamentally changes the entanglement structure despite leaving the gate count and depth identical.

4.3. Quantum Properties of Dataset

- **Qubit range:** 1–20 qubits, covering single-qubit toy circuits up to large random circuits requiring substantial entanglement reasoning.
- **Circuit depth:** 0–681 layers across all programs, with structured algorithms (Bell State, GHZ) occupying

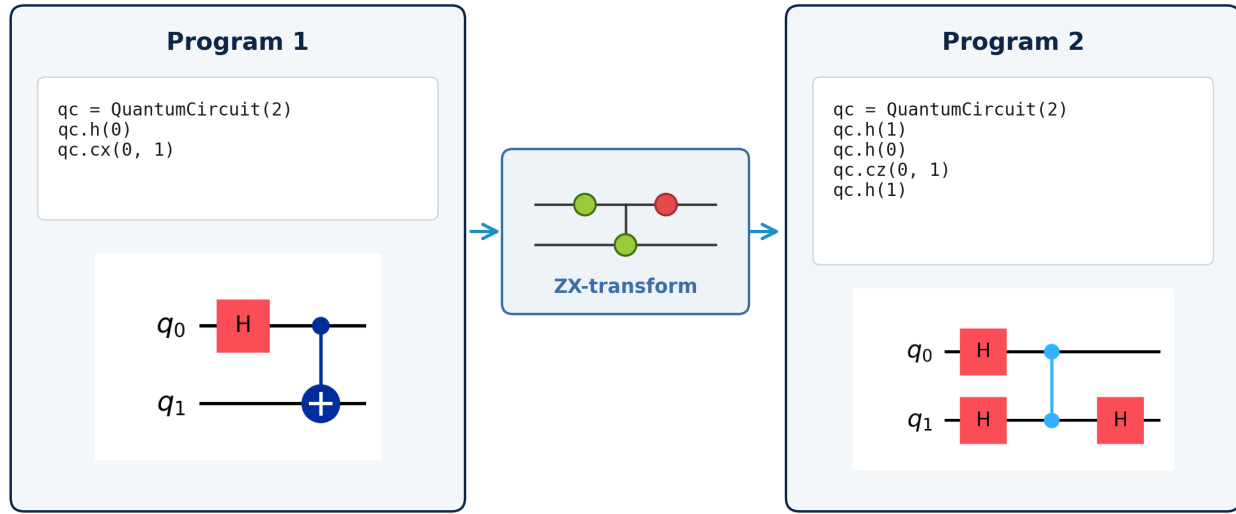


Figure 2. Semantically equivalent circuit pair represented by code and circuit, generated by ZX-transform

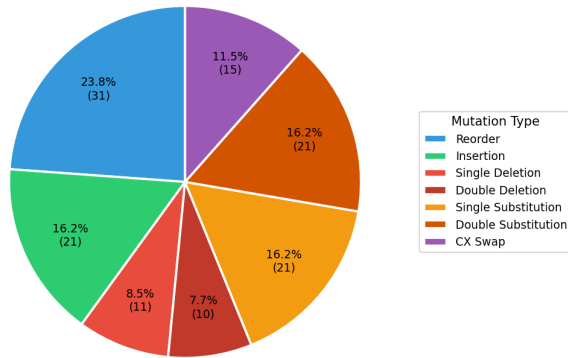


Figure 3. Distribution of mutation types across the 130 non-equivalent circuit pairs. Reorder mutations are the most frequent (23.8%), while CX Swap variants constitute the smallest group (11.5%).

the shallow end and ZX-optimized random circuits reaching the upper extreme.

- **Gate set:** 44 distinct Qiskit gate types, spanning single-qubit gates (rx, ry, rz, sx, sdg, tdg), standard two-qubit gates (cx, cz, cy, cp, swap, iswap), controlled rotations (crx, cry, crz, cu), multi-qubit gates (ccx, ccz, cswap, rccx), and exotic two-qubit interactions (xx_plus_yy, xx_minus_yy, ecr, dcx). The most frequent gates are cz (3,249 occurrences), rz (2,611), and cx (2,451), reflecting the prevalence of ZX-calculus rewriting which favours CZ/RZ decompositions.

- **Total size:** 260 labeled pairs (130 equivalent, 130 non-equivalent).
- **Verification:** Equivalent pairs are verified via ZX-calculus identity reduction and confirmed by statevector fidelity $F = 1$; non-equivalent pairs are verified by computing $F < 1$ on at least one computational basis input state via full unitary comparison.

This construction allows systematic evaluation of LLMs' reasoning over semantic equivalence in quantum programs across a diverse range of circuit structures, sizes, and gate vocabularies.

5. Experimental Setup

All models are queried using a zero-shot prompt consisting of a **system prompt** and a **sample data point**, with temperature fixed at 0.0 to ensure deterministic outputs.

5.1. System Prompt

The system prompt establishes the task context and enforces a strict structured output format:

You are an expert in quantum computing and Qiskit circuit analysis. Your task is to determine whether two quantum circuits are semantically equivalent, meaning they implement the same unitary transformation on their qubits.

Input format:

Program 1: { <Qiskit code> }

```

Program 2: { <Qiskit code> }

Respond with ONLY valid JSON, no
explanation, no markdown:
{"prediction": true} or
{"prediction": false}
    
```

A sample data is provided in the following :

```

Program 1:
{
<Qiskit source code of circuit 1>
}

Program 2:
{
<Qiskit source code of circuit 2>
}
    
```

No chain-of-thought, few-shot examples, or domain hints are provided. The model must determine equivalence solely from the raw Qiskit source code, making this a direct test of the model's capacity for semantic reasoning over quantum programs.

5.2. Model Under Evaluation

We evaluate 13 large language models encompassing a diverse range of architectures, scales, and access paradigms. This set includes five proprietary models (GPT-4o, GPT-4.1, o4-mini, Gemini 2.5 Pro, Grok 4.3) and eight open-weight models (LLaMA 3.3 70B, DeepSeek-V3, Mixtral 8x22B, Mistral 7B, Qwen 2.5 72B, Qwen 3 235B, GPT-OSS 120B), with parameter counts ranging from 7.3B to 671B. Models were accessed through four inference providers: OpenAI API, Google AI Studio, OpenRouter, and Cerebras/Groq and all queries were performed at a temperature setting of 0.0 to ensure reproducibility. This selection deliberately contrasts state-of-the-art closed-source systems with open-weight alternatives, thereby enabling a rigorous comparison of reasoning capabilities across model families on the quantum semantic equivalence benchmark.

5.3. Evaluation Metrics

Since the two evaluation phases are structurally asymmetric, Phase I contains exclusively equivalent pairs (ground truth = true) and Phase II contains exclusively non-equivalent pairs (ground truth = false), a single aggregate accuracy score would conflate two distinct reasoning abilities. We therefore report accuracy separately for each phase.

For Phase I, accuracy corresponds to **recall** (true positive rate): the fraction of equivalent circuit pairs that the model correctly identifies as equivalent. For Phase II, accuracy corresponds to **specificity** (true negative rate): the fraction of non-equivalent pairs that the model correctly rejects. In both

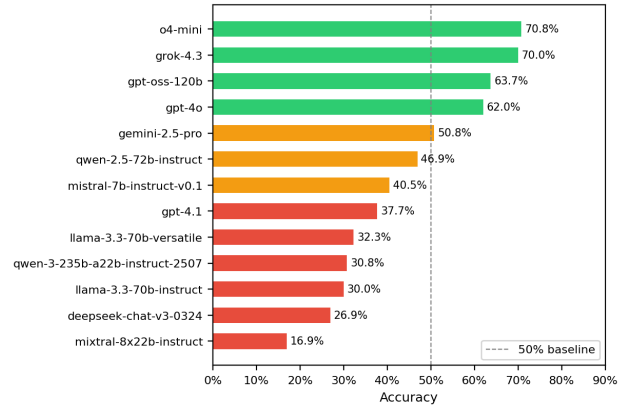


Figure 4. Accuracy of the models for the Semantic Equivalent Pairs

cases, a random baseline produces an expected accuracy of **50%**, as the task is binary.

In addition to accuracy, we trace **token usage** (prompt tokens and completion tokens) for each model call, enabling a cost-efficiency analysis that relates model performance to inference cost across providers.

6. Results and Discussions

In this section, we present the evaluation results of large language models on the Quantum-SemEval benchmark, analyzing performance on both semantically equivalent and non-equivalent circuit pairs.

6.1. Semantic Equivalent Pairs

Only four models surpass the 50% random baseline: o4-mini (70.8%), grok-4.3 (70.0%), gpt-oss-120b (63.7%), and gpt-4o (62.0%) — indicating that recognising semantic equivalence under ZX-calculus transformation remains challenging even for frontier models. The majority of open-weight models cluster well below baseline, with mixtral-8x22b scoring only 16.9%, suggesting a strong bias toward predicting false (non-equivalent). Notably, no model achieves accuracy above 71%, highlighting that the syntactic divergence introduced by ZX-calculus rewriting effectively prevents trivial pattern-matching solutions.

6.2. Semantic Non-Equivalent Pairs

Nearly all evaluated models exceed the 50% random baseline by a wide margin: gpt-oss-120b achieves perfect accuracy (100%), gemini-2.5-pro reaches 99.2%, and seven of the nine evaluated models score above 80%, suggesting that structural mutations such as gate reordering, gate substitutions, additions, deletions and control-target

Table 2. Large language models evaluated in this study. **OW** = Open-weight; **CS** = Closed-source (proprietary). [†]Parameter count not publicly disclosed. [‡]Mixture-of-Experts; active parameters shown in parentheses.

Model	Provider	Parameters	Type	API / Inference
<i>Proprietary Models</i>				
GPT-4o	OpenAI	[†]	CS	OpenAI API
GPT-4.1	OpenAI	[†]	CS	OpenAI API
o4-mini	OpenAI	[†]	CS	OpenAI API
Gemini 2.5 Pro	Google	[†]	CS	Google AI Studio
Grok 4.3	xAI	[†]	CS	OpenRouter
<i>Open-weight Models</i>				
GPT-OSS 120B	OpenAI	120B	OW	Cerebras
LLaMA 3.3 70B	Meta	70B	OW	Groq
LLaMA 3.3 70B Instruct	Meta	70B	OW	OpenRouter
DeepSeek-V3 (0324)	DeepSeek	671B [‡] (37B)	OW	OpenRouter
Mixtral 8×22B	Mistral AI	141B [‡] (39B)	OW	OpenRouter
Mistral 7B v0.1	Mistral AI	7.3B	OW	OpenRouter
Qwen 2.5 72B	Alibaba	72B	OW	OpenRouter
Qwen 3 235B	Alibaba	235B [‡] (22B)	OW	Cerebras

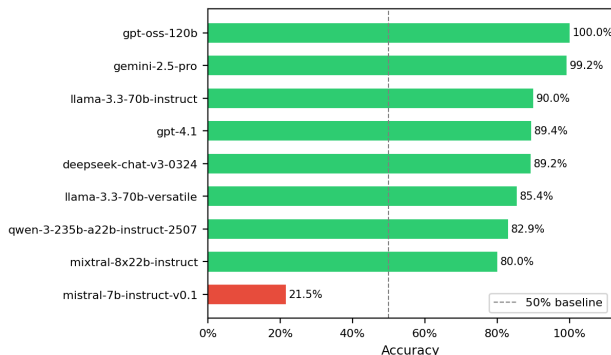


Figure 5. Accuracy of the models for the Semantic Non-Equivalent Pairs

swaps — introduce sufficient syntactic difference for models to detect non-equivalence without deep semantic reasoning, effectively making this phase a surface-level pattern recognition task rather than a true test of quantum circuit understanding.

6.3. Overall Result

Across both evaluation phases, the results reveal a consistent and fundamental asymmetry in how LLMs reason about quantum circuit semantics. On the equivalent circuit phase, performance is uniformly low, only four models surpass the 50% random baseline, with the best model (o4-mini) reaching just 70.8%, indicating that confirming semantic equivalence under ZX-calculus transformation is a genuinely hard task for current LLMs. On the non-equivalent phase, the picture reverses: eight of the nine evaluated models exceed 80%, suggesting that detecting non-equivalence is comparatively straightforward, as the structural mutations,

gate substitutions, deletions, and reorderings introduce sufficient syntactic dissimilarity for models to identify the circuits as different without requiring deep semantic reasoning.

No single model performs strongly across both phases simultaneously. Models that excel at detecting non-equivalence (gpt-oss-120b: 100%, gemini-2.5-pro: 99.2%) score modestly on equivalence detection (63.7% and 50.8% respectively), while the top performers on equivalence (o4-mini: 70.8%, grok-4.3: 70.0%) could not be evaluated on Phase II due to budget constraints. mistral-7b-instruct-v0.1 stands out as the most extreme case of this imbalance, scoring 40.5% in Phase I and only 21.5% in Phase II, revealing an almost exclusive bias toward predicting `true`.

These findings suggest that LLMs lack genuine semantic understanding of quantum circuits and instead rely on surface-level syntactic features. The ZX-calculus transformation, which substantially increases code length while preserving semantics, appears to be highly effective at exposing this limitation of large language models.

7. Limitations and Future Directions

QuantumSemEval is a first step toward measuring LLM reasoning over quantum program semantics, and several aspects of its current construction limit the scope of what it can presently measure.

- **Equivalence oracle is ZX-based.** The equivalent pairs in our dataset are generated by applying ZX-calculus simplification (e.g., the `full_reduce` pass in PyZX) to a source circuit and pairing the original with its reduced form (Peham et al., 2022b; Duncan et al., 2019).

Model	Semantic Equivalent		Non-Equivalent	
	Accuracy (%)	Tokens (K)	Accuracy (%)	Tokens (K)
o4-mini	70.8	394	—*	—
Grok 4.3	70.0	273	—*	—
GPT-OSS 120B	63.7	260	100.0	210
GPT-4o	62.0	150	—*	—
Gemini 2.5 Pro	50.8	257	99.2	158
Qwen 2.5 72B	46.9	165	—*	—
Mistral 7B v0.1	40.5	137	21.5	105
GPT-4.1	37.7	151	89.4 [†]	43
LLaMA 3.3 70B	32.3	154	85.4	79
Qwen 3 235B	30.8	166	82.9	82
LLaMA 3.3 70B Instruct	30.0	153	90.0	76
DeepSeek-V3	26.9	153	89.2	77
Mixtral 8×22B	16.9	188	80.0	93

Table 3. Model accuracy and total token consumption across both evaluation phases. Bold denotes the best accuracy per phase.

[†]Partial evaluation (85/130 samples completed). *Could not be evaluated due to API budget constraints.

While this approach guarantees that labeled equivalent pairs are truly semantically identical under the ZX rules, it introduces a bias toward equivalences that the ZX normal-form reduction can discover. Equivalences requiring identities outside the ZX-calculus, such as those involving Solovay–Kitaev style approximations or rewrites that introduce ancilla qubits, are under-represented. Similarly, methods based on decision diagrams and tensor network equivalence checking have complementary strengths but are not captured by the current pipeline (Peham et al., 2022a). Future versions should combine ZX-based reasoning with other equivalence checkers (e.g., decision-diagram approaches and peephole rewriting) to broaden the space of equivalences sampled.

- **Non-equivalence is constructed from a fixed transformation set.** Expanding the perturbation vocabulary, and including adversarial perturbations that preserve the unitary on the computational-basis input but differ elsewhere, is an important next step.
- **Scale and gate-set scope.** The benchmark currently spans circuits with 2 to 20 qubits and depths of 5 to 40, drawn from a restricted standard-gate set (Clifford+T plus controlled rotations and a few two-qubit gates). It does not yet include mid-circuit measurement, classical control, reset, parameterised circuits with symbolic parameters, or larger-scale circuits where state-vector verification is infeasible. Scaling the verification pipeline, for example via tensor-network or decision-diagram backends, would allow the benchmark to reach the regimes where LLM-assisted reasoning matters most.
- **Exact, noiseless equivalence only.** Equivalence labels are assigned by exact unitary comparison

(`Operator.equiv`) and statevector fidelity, on idealised noiseless circuits. This is deliberate: we view semantic equivalence as a property of the program, not the device. However, it means the benchmark does not measure whether LLMs can reason about approximate equivalence, equivalence up to noise, or equivalence on real NISQ hardware. A future extension could add a tolerance-graded variant in which pairs are labelled by total-variation distance or diamond-norm distance rather than a hard boolean.

- **Algorithm coverage is broad but not exhaustive.** The current pool includes Bell, GHZ, and W states, Bernstein–Vazirani oracles, Grover oracles, MAJ-3, Toffoli, Fredkin, CCH, the inverse QFT, bit- and phase-flip codes, ripple-carry adders, simple error correction and full-entanglement circuits, together with random Clifford+T circuits of varying depth. Important families are still missing, in particular variational ansatz circuits (VQE, QAOA, hardware-efficient ansatz) with concrete parameter values, quantum-error-correction stabiliser circuits beyond toy codes, and circuits over multiple registers with structured data layouts. Adding these would test whether LLM reasoning generalises beyond the textbook-style circuits that dominate web-scale training data.
- **Dataset size.** Our initial release contains on the order of a few hundred labelled pairs across the equivalent and non-equivalent splits. This is enough to expose systematic failure modes in current frontier models, but too small for fine-grained training or for statistically tight per-category accuracy estimates. Programmatic scale-up of the generation pipeline is straightforward and is a primary direction for the next release.

References

- Badihi, S., Li, Y., and Rubin, J. Eqbench: A dataset of equivalent and non-equivalent program pairs. In *Proceedings of the 18th Mining Software Repositories Conference (MSR)*, pp. 610–614, 2021. doi: 10.1109/MSR52588.2021.00084.
- Basit, A., Shao, M., Asif, M. H., Innan, N., Kashif, M., Marchisio, A., and Shafique, M. Qhackbench: Benchmarking large language models for quantum code generation using pennylane hackathon challenges. In *2025 IEEE International Conference on Quantum Artificial Intelligence (QAI)*, pp. 316–322. IEEE, 2025.
- Calude, C. S., Dinneen, M. J., and Hua, R. Quantum solutions for densest k-subgraph problems. *Journal of Membrane Computing*, 2(1):26–41, 2020.
- Ciacco, A., Guerriero, F., and Saccomanno, F. P. Quantum annealing for the two-level facility location problem. *Future Generation Computer Systems*, 174:107961, 2026.
- Duncan, R., Kissinger, A., Perdrix, S., and van de Wetering, J. Graph-theoretic simplification of quantum circuits with the zx-calculus. *Quantum*, 4:279, 2019.
- Gebler, C., Rethmann, J., and Ueberholz, P. Qubo models for the fifo stack-up problem and experimental evaluation on a quantum annealer. *SN Computer Science*, 5(7):818, 2024.
- Guo, X., Wang, M., and Zhao, J. Quanbench: Benchmarking quantum code generation with large language models. *arXiv preprint arXiv:2510.16779*, 2025.
- Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., et al. Quantum computing with qiskit. *arXiv preprint arXiv:2405.08810*, 2024.
- Kissinger, A. and Van De Wetering, J. Pyzx: Large scale automated diagrammatic reasoning. *arXiv preprint arXiv:1904.04735*, 2019.
- Marini, R. and Arpe, R. Cyber risk scoring with qubo: A quantum and hybrid benchmark study. *arXiv preprint arXiv:2512.18305*, 2025.
- Maveli, N., Vergari, A., and Cohen, S. B. What can large language models capture about code functional equivalence? In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 6880–6918, 2025. doi: 10.18653/v1/2025.findings-naacl.382.
- Moncayo-Martínez, L. A. and He, N. Quantum optimisation for supply chain: Qubo formulations and qaoa solutions for facility location and load balancing. *Results in Engineering*, pp. 108373, 2025.
- Mücke, S., Heese, R., Müller, S., Wolter, M., and Pi- atkowski, N. Feature selection on quantum computers. *Quantum Machine Intelligence*, 5(1):11, 2023.
- Negre, C. F., Lopez-Bezanilla, A., Zhang, Y., Akrobotu, P. D., Mniszewski, S. M., Tretiak, S., and Dub, P. A. Toward a qubo-based density matrix electronic structure method. *Journal of chemical theory and computation*, 18(7):4177–4185, 2022.
- Nielsen, M. A. and Chuang, I. L. *Quantum computation and quantum information*. Cambridge university press, 2010.
- Pathak, I. P., Kim, T., Yilmazer, N., and Roach, D. Feature selection for imbalanced network intrusion detection systems dataset using quantum annealer. In *2025 IEEE International Conference on Electro Information Technology (eIT)*, pp. 1–6. IEEE, 2025.
- Peham, T., Burgholzer, L., and Wille, R. Equivalence checking paradigms in quantum circuit design: A case study. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pp. 517–522, 2022a. doi: 10.1145/3489517.3530480.
- Peham, T., Burgholzer, L., and Wille, R. Equivalence checking of quantum circuits with the zx-calculus. *arXiv preprint arXiv:2208.12820*, 2022b.
- Preskill, J. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.
- Samanta, S., Wang, D., Hodges, T., and Ferenczi, A. Parametric quantum feature selection methods for fraud and default detection. In *Proceedings of the AAAI Symposium Series*, volume 7, pp. 390–397, 2025.
- Shikuri, Y. Decomposed quadratization: efficient qubo formulation for learning bayesian network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 11345–11352, 2025.
- Slim, A., Hamieh, H., Kotaich, J., Ghosn, Y., Chehimi, M., Mohanna, A., Hammoud, H. A. A. K., and Ghanem, B. Quanbench+: A unified multi-framework benchmark for llm-based quantum code generation, 2026. URL <https://arxiv.org/abs/2604.08570>.
- Vishwakarma, S., Harkins, F., Golecha, S., Bajpe, V. S., Dupuis, N., Buratti, L., Kremer, D., Faro, I., Puri, R., and Cruz-Benito, J. Qiskit humaneval: An evaluation benchmark for quantum code generative models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pp. 1169–1176. IEEE, 2024.

Wei, A., Cao, J., Li, R., Chen, H., Zhang, Y., Wang, Z.,
Liu, Y., Teixeira, T. S. F. X., Yang, D., and Aiken, A.
Equibench: Benchmarking large language models' under-
standing of program semantics via equivalence checking.
arXiv, abs/2502.12466, 2025.

Wong, T. G. *Introduction to classical and quantum comput-
ing*. Rooted Grove Omaha, Nebraska, 2022.

Zhou, Z., Chen, Q., Zhang, C., Tan, M., and Li, S. Hybrid
quantum annealing for large-scale exam scheduling: Vali-
dation in real-world educational scenarios. *Applied Soft
Computing*, pp. 113756, 2025.